# A-DAPT Blueprint

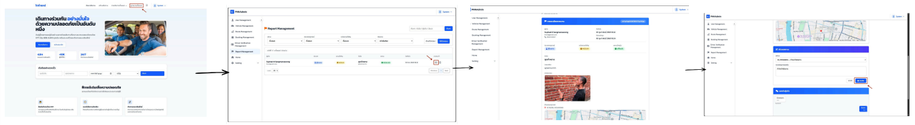
## CUSTOMER VISIBLE

### ACCEPTANCE TEST

### USER INTERFACE (UI)

User

Admin



## CUSTOMER INVISIBLE

### BUSINESS LOGIC

การเข้าสู่ระบบ (Authentication Logic)
- ผู้ใช้พิมพ์ Login ก่อนใช้งานระบบ
- ระบบตรวจสอบ Email + Password กับฐานข้อมูล
- ถ้าถูกต้อง → สร้าง Session / Token ให้ใช้งานต่อ
- ระบบจะรู้ว่าเป็น
  - User ปกติ
  - หรือ Admin
- สิทธิ์การเข้าถึงข้อมูลและหน้าต่างต่างกันตาม Role

การจัดการรายงานโดย Admin (Admin Management Logic)
Admin เปิดหน้า /admin/reports
ระบบจะ:
ดึงรายงานทั้งหมด
พร้อมรองรับ:
- Filter ตาม Status
- Filter ตาม Incident Type
- Filter ตาม Priority
- Search keyword
- Pagination

เพื่อให้จัดการข้อมูลจำนวนมากได้

การสร้างรายงานเหตุการณ์ (Create Incident Report)
เมื่อ User แจ้งเหตุ ระบบทำงานตามลำดับนี้:
Step 1 — รับข้อมูลจากฟอร์ม
User ต้องกรอก:
- ประเภทเหตุการณ์ (Incident Type)
- ชื่อเรื่อง
- รายละเอียด
- ระดับความสำคัญ
- ตำแหน่ง(ละติจูด-ลองจิจูด เป็นต้น)
- แนบรูปภาพ (ถ้ามี)
- ถ้าข้อมูลไม่ครบ ระบบจะไม่ให้ส่ง

Admin ตรวจสอบรายละเอียด
เมื่อเหตุการณ์ถูกระบบโหลด:
- ข้อมูลผู้แจ้ง
- รายละเอียดเหตุการณ์
- รูปภาพ (Lightbox)
- แผนที่ตำแหน่งเกิดเหตุ
ใช้ location ที่ user บักหมุดมาแสดงแบบ read-only

การบริหารสถานะของเหตุการณ์ (Workflow Processing)
Admin สามารถเปลี่ยนสถานะได้ 4 แบบ:
Status ทางการของเหตุ
PENDING
รอดำเนินการ
IN_PROGRESS
กำลังตรวจสอบ
RESOLVED
แก้ไขแล้ว
REJECTED
ปฏิเสธ
โดยการเปลี่ยนสถานะ Admin จะ:
- PATCH Update สถานะใน Database
- เพิ่ม Admin Note
- User จะเห็นสถานะใหม่ทันที
ทำให้เกิด Workflow การจัดการปัญหา

จัดการไฟล์รูปภาพ
ถ้าการเก็บไฟล์เหตุ:
- รองรับ Upload ไปเก็บที่ Cloudinary
- เก็บแค่ชื่อ URL ของรูปใน Database
- ไม่เก็บไฟล์จริงใน Server
- ลดภาระ Server และทำให้โหลดเร็ว

Step 2 — Validate ข้อมูลก่อนบันทึก
Backend ตรวจสอบ:
- ความยาวข้อความเป็นไปตามที่กำหนด
- ชื่อ location ไม่ว่างเปล่า
- โทรศัพท์เป็นเลขใน SMS
- ประเภทเหตุการณ์ต้องอยู่ใน ENUM ที่กำหนด

ENUM คือ:
- DRIVER, PASSENGER, ROUTE, BOOKING, SYSTEM, OTHER

บันทึกข้อมูลลง Database
Backend จะสร้าง Record ใหม่ในตาราง Report โดยมี:
- incidentType
- priority
- description
- location (lat,lng,address)
- imageUrl
- status = PENDING (ค่าเริ่มต้น)

รายงานใหม่จะปรากฏในระบบทันทีผ่าน API "กลุ่มรายงานการ"

การแสดงผลรายงานของผู้ใช้ (User View Logic)
เมื่อ User เปิดหน้า My Reports
ระบบจะ:
- Query รายงานเฉพาะของ user คนนั้น
- แสดง:
  - ประเภทเหตุการณ์
  - ระดับความสำคัญ
  - สถานะ
  - ข้อความจาก Admin
  - รูปภาพ + แผนที่

User เห็นผลการทำงานของตัวเองโดยไม่เห็นของคนอื่น

การลบรายงาน (Delete Logic)
ถ้า Admin ลบรายงาน:
- ระบบจะ Record ค่าที่ Database ยกให้
- รายงานจะไม่สามารถดูได้อีกได้

### DATABASE

User

| Field | Type |
|---|---|
| userId | INT (PK) |
| name | VARCHAR |
| email | VARCHAR |
| role | VARCHAR |

1 → M

Report

| Field | Type |
|---|---|
| reportId | INT (PK) |
| incidentType | VARCHAR |
| title | VARCHAR |
| description | TEXT |
| latitude | DECIMAL |
| longitude | DECIMAL |
| status | VARCHAR |
| updatedAt | VARCHAR |
| createdAt | DATETIME |
| imageUrl | VARCHAR |

1 → M

ReportAttachment

| Field | Type |
|---|---|
| attachmentId | INT (PK) |
| reportId | INT (FK) |
| fileUrl | VARCHAR |
| uploadedAt | DATETIME |

### SUPPORT

Authentication & Security
- ระบบมีการ Login ก่อนใช้งาน (Authentication Required)
- ใช้การตรวจสอบตัวตนผ่าน Email / Password
- มีการแยกสิทธิ์ผู้ใช้:
  - User → แจ้งเหตุ / ดูรายงานตัวเอง
  - Admin → จัดการรายงานทั้งหมด
- ป้องกันการเข้าถึง API โดยไม่ได้ Login
- ใช้ Token / Session เพื่อรักษาสถานะการใช้งาน

ทำให้ข้อมูลเหตุการณ์ไม่ถูกเข้าถึงจากบุคคลภายนอก

ORM (Object-Relational Mapping)
- ใช้ Prisma ORM จัดการ Database
- Backend ไม่ต้องเขียน SQL ตรง ๆ
- แปลงคำสั่งจาก Code → Query Database อัตโนมัติ
- ช่วยให้:
  - ลด Bug จาก SQL
  - โครงสร้างข้อมูลชัดเจน
  - แก้ไข Schema ได้ง่าย
- ใช้ Prisma Model สำหรับ Report, User, Location ฯลฯ

ทำให้ระบบพัฒนาเร็วและ Maintain ง่าย

RESTful API
- Backend ให้บริการผ่าน REST API
- Frontend เรียกใช้งานผ่าน HTTP เช่น:
  - POST /reports → สร้าง Incident
  - GET /reports → ดึงข้อมูลรายงาน
  - PATCH /reports/:id → อัปเดตสถานะ
- API เป็นตัวกลางเชื่อม:
- Frontend (UI) ⇄ Backend Logic ⇄ Database

ทำให้ระบบแยกส่วน (Decoupled Architecture)

File Storage (Cloudinary)
- ระบบ ไม่เก็บรูปใน Server
- Upload รูปไปที่ Cloudinary
- Database เก็บเฉพาะ URL
- ลดภาระ Server และเพิ่มความเร็วในการโหลด

รองรับการแนบหลักฐานเหตุการณ์ได้จริง

Map & Location Service
- ใช้ระบบพิกัด Latitude / Longitude
- ผู้ใช้ต้องปักหมุดก่อนส่งรายงาน
- Backend บันทึกตำแหน่งเพื่อให้ Admin ตรวจสอบบนแผนที่ได้

ทำให้ Incident มีตำแหน่งจริง ตรวจสอบได้